

MAIL AGENT

Agentic Email & Calendar Operations Platform

Product & Engineering Build Guide

Phase 1: Gmail + Google Calendar | Phase 2: Microsoft Outlook / 365

Version 1.0 • June 2026

Table of Contents

1. Executive Summary
2. Product Scope & Design Principles
3. System Architecture — High Level
4. Multi-Agent Design with LangGraph
5. Agent Roster & Responsibilities
6. Tool Layer — Schemas & Contracts
7. Permission & Approval System
8. Database Schema (PostgreSQL)
9. Backend Architecture
10. Frontend Dashboard Specification
11. Gmail + Google Calendar Integration (Phase 1)
12. Phase 2: Microsoft Outlook / 365 Migration Plan
13. Formatting & Style Engine
14. Observability, Testing & Evals
15. Security & Compliance
16. Build Roadmap
17. Appendix: Technology Stack Summary

1. Executive Summary

Mail Agent is a multi-agent AI system that lets a user manage their entire email and calendar workflow through natural-language instructions. Instead of clicking through inbox filters, the user types what they want — "summarize today's unread emails from clients and draft replies in a formal tone with my signature" — and a team of specialized AI agents plans the work, executes it through governed tools, and surfaces results in a control dashboard.

The system is built as a supervisor/worker multi-agent graph using LangGraph, with each worker agent owning one capability domain (reading, categorizing, drafting, sending, scheduling, reminders). Agents act through a strict tool layer with a permission system in front of every side-effecting action (send email, create event, modify calendar) so that nothing irreversible happens without the right authorization level — by default, user confirmation.

Phase 1 ships against Gmail and Google Calendar, since Google's APIs are the most directly accessible and well documented for this use case. Phase 2 extends the same agent and tool abstractions to Microsoft Outlook / Graph API, reusing close to the entire orchestration layer — only the integration adapter changes.

1.1 What Success Looks Like

- A user can issue a single free-text instruction and have it correctly decomposed into the right sequence of agent actions.
- Every email categorization, summary, and draft is explainable — the user can see why an agent made a decision.
- No email is sent and no calendar event is created without passing through the permission layer.
- Formatting instructions (signature, tone, font, color, spacing) are honored precisely and consistently.
- The architecture is provider-agnostic enough that adding Outlook in Phase 2 does not require rebuilding the agent layer.

2. Product Scope & Design Principles

2.1 Core Capabilities (Phase 1)

Capability	Description
Read & Search	Fetch emails by query, label, sender, date range, thread
Categorize	Auto-label emails (e.g. Urgent, FYI, Meeting Request, Newsletter, Action Needed)
Summarize	Single email or full thread summarization, on demand or scheduled
Draft Replies	Context-aware draft generation honoring tone, formatting, and signature instructions
Send	Send a drafted or user-approved email, with permission gating
Reminders	Create follow-up reminders/tasks tied to specific emails or threads
Meetings	Detect meeting intent in emails and create Google Calendar events, with conflict checking
Dashboard Control	Web UI to view agent activity, approve/reject actions, configure permissions and rules

2.2 Design Principles

1. Agent autonomy is opt-in, not default. Every agent starts conservative (draft + confirm) and the user can grant more autonomy per action type over time.
2. Every action is auditable. Every tool call, decision, and state transition is logged with the reasoning that produced it.
3. Tools are the only way agents touch the world. No agent calls the Gmail API directly from an LLM completion — it always goes through a typed, validated tool function.
4. Formatting is a parameter, not a vibe. Style instructions (font, color, indentation, signature) are parsed into a structured style object, not left to free-text generation alone.

- 5. Provider-agnostic core. The agent graph, permission system, and database schema are designed around abstract concepts (Message, Thread, Event, Contact) that both Gmail and Outlook adapters implement.

3. System Architecture — High Level

The system has five layers. Each is independently testable and replaceable.

Layer	Responsibility	Key Tech
Presentation	Chat interface + dashboard for agent control, approvals, and configuration	React, WebSockets
Orchestration	Multi-agent reasoning, planning, task delegation, state machine	LangGraph, LangChain, Claude API
Tool / Integration	Typed wrappers around Gmail, Calendar, Tasks, and (later) Outlook APIs	Python, Google API Client, MS Graph SDK
Permission & Policy	Authorization checks, approval queue, rule engine	Custom policy engine + Postgres
Persistence	Conversation state, email metadata cache, audit log, user config	PostgreSQL, Redis (cache/queue)

3.1 Request Flow (Happy Path)

1. User types an instruction in the dashboard chat panel.
2. Request hits the FastAPI backend, which loads conversation + user context and invokes the LangGraph supervisor.
3. Supervisor agent decomposes the instruction into subtasks and routes each to the relevant worker agent (Reader, Categorizer, Drafter, Scheduler, Reminder).
4. Worker agents call tools (Gmail/Calendar wrappers). Every side-effecting tool call is intercepted by the Permission Layer first.

5. If the action requires approval, it is written to an approval queue and surfaced in the dashboard; execution pauses (LangGraph interrupt) until the user approves or rejects.
 6. Once approved (or if pre-authorized), the tool executes against the Gmail/Calendar API.
 7. Results flow back through the supervisor, get logged to the audit trail, and are streamed to the dashboard UI.
-

4. Multi-Agent Design with LangGraph

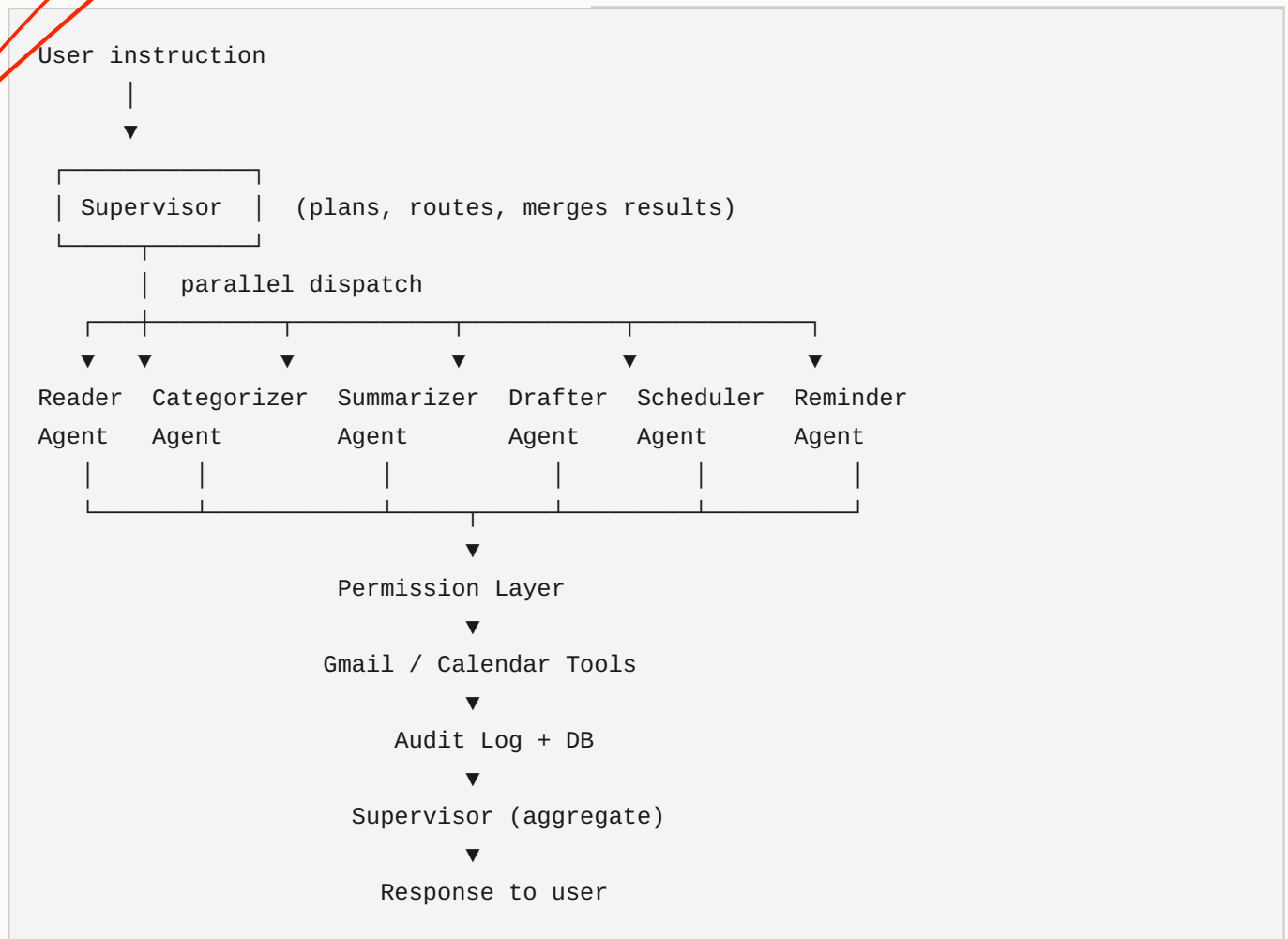
LangGraph models the system as a graph of nodes (agents/tools) and edges (control flow), with a shared state object passed between nodes. This gives you parallel execution, conditional routing, and human-in-the-loop interrupts — all of which this product needs.

4.1 Why Multi-Agent Instead of One Big Agent

- Separation of concerns: a categorization agent and a drafting agent need very different prompts, context windows, and evaluation criteria. Mixing them in one mega-prompt degrades quality on both.
- Parallelism: categorizing 50 unread emails and summarizing 5 threads can run concurrently as independent graph branches rather than serially.
- Safety isolation: only the agents that need send/create permissions are wired to those tools. A summarization agent simply has no tool access to `send_email` — that's a structural guarantee, not a prompt instruction.
- Easier evals: you can test "does the Categorizer agent assign correct labels" independently of "does the Drafter agent match tone."

4.2 Supervisor Pattern

A Supervisor (Orchestrator) agent sits at the root of the graph. It does not execute domain work itself — it interprets the user's natural-language request, breaks it into a task list, and routes each task to the correct worker node. This is the standard LangGraph "supervisor" multi-agent pattern.



4.3 LangGraph State Object

All agents read from and write to a shared, typed state object. This is the contract that keeps multi-agent coordination sane.

```
from typing import TypedDict, Literal, Optional
from langgraph.graph import MessagesState

class MailAgentState(MessagesState):
    user_id: str
    instruction: str                # raw user request
    plan: list[dict]                # supervisor's decomposed task list
    active_tasks: list[dict]        # tasks currently being worked
    completed_tasks: list[dict]
    pending_approvals: list[dict]   # actions awaiting user confirmation
    email_context: list[dict]       # fetched emails/threads in scope
    draft_results: list[dict]
    calendar_results: list[dict]
    style_profile: dict             # signature, tone, font/color prefs
    errors: list[dict]
```


4.4 Example Graph Definition (Python)

```

from langgraph.graph import StateGraph, START, END
from langgraph.types import interrupt

graph = StateGraph(MailAgentState)

graph.add_node("supervisor", supervisor_node)
graph.add_node("reader", reader_agent_node)
graph.add_node("categorizer", categorizer_agent_node)
graph.add_node("summarizer", summarizer_agent_node)
graph.add_node("drafter", drafter_agent_node)
graph.add_node("scheduler", scheduler_agent_node)
graph.add_node("reminder", reminder_agent_node)
graph.add_node("permission_gate", permission_gate_node)
graph.add_node("executor", tool_executor_node)
graph.add_node("aggregator", aggregator_node)

graph.add_edge(START, "supervisor")

# Supervisor conditionally routes to one or many workers in parallel
graph.add_conditional_edges(
    "supervisor",
    route_to_workers,  # returns list of node names to fan out to
    ["reader", "categorizer", "summarizer", "drafter", "scheduler", "reminder"]
)

for worker in ["reader", "categorizer", "summarizer", "drafter", "scheduler",
               "reminder"]:
    graph.add_edge(worker, "permission_gate")

graph.add_conditional_edges(
    "permission_gate",
    needs_human_approval,  # checks action risk level
    {
        "approve_required": END,  # interrupt() pauses graph, waits on dashboard
        "auto_approved": "executor"
    }
)

graph.add_edge("executor", "aggregator")
graph.add_edge("aggregator", END)

app = graph.compile(checkpointer=postgres_checkpointer)  # enables resume after
interrupt

```

Note: LangGraph's `interrupt()` + checkpointer pattern is what makes "pause for human approval, then resume exactly where it left off" possible without re-running completed work. Use a Postgres-backed checkpointer in production so interrupted runs survive backend restarts.

5. Agent Roster & Responsibilities

Agent	Role	Tools It Can Call	Output
Supervisor	Parses user intent, builds task plan, routes work, merges results	None (orchestration only)	Task plan + final response
Reader	Fetches emails/threads matching query/filter criteria	<code>list_emails</code> , <code>get_thread</code> , <code>get_attachment</code>	Email/thread objects
Categorizer	Assigns category labels using rules + LLM judgment	<code>list_labels</code> , <code>apply_label</code>	Label assignments
Summarizer	Produces concise summaries of single emails or full threads	<code>get_thread</code> (read-only)	Summary text
Drafter	Generates reply drafts honoring tone/style/signature instructions	<code>get_thread</code> , <code>get_style_profile</code> , <code>create_draft</code>	Draft object (not sent)
Sender	Sends a previously created and approved draft	<code>send_email</code> (gated)	Send confirmation
Scheduler	Detects meeting intent, checks calendar conflicts, creates events	<code>check_availability</code> , <code>create_event</code> (gated)	Calendar event
Reminder	Creates follow-up reminders/tasks linked to emails	<code>create_task</code> , <code>create_reminder</code>	Task object

Note: Sender and Scheduler are the only agents wired to irreversible, externally-visible actions (`send_email`, `create_event`). They are intentionally the most tightly gated agents in the system — see Section 7.

6. Tool Layer — Schemas & Contracts

Tools are the boundary between LLM reasoning and real-world effects. Every tool has a strict input schema (Pydantic), explicit side-effect classification (read-only / reversible / irreversible), and a permission tag consumed by the Permission Layer.

6.1 Tool Definitions (Python / Pydantic + LangChain @tool)

```

from pydantic import BaseModel, Field
from langchain_core.tools import tool
from enum import Enum

class SideEffect(str, Enum):
    READ_ONLY = "read_only"
    REVERSIBLE = "reversible"          # e.g. create_draft, apply_label
    IRREVERSIBLE = "irreversible"      # e.g. send_email, create_event

class ListEmailsInput(BaseModel):
    query: str = Field(..., description="Gmail search query syntax, e.g. 'from:boss is:unread'")
    max_results: int = Field(default=20, le=100)

@tool("list_emails", args_schema=ListEmailsInput)
def list_emails(query: str, max_results: int = 20) -> list[dict]:
    """Search and list emails matching a Gmail query. Read-only."""
    return gmail_client.search(query=query, max_results=max_results)

class CreateDraftInput(BaseModel):
    thread_id: str
    body_markdown: str
    style_profile_id: str | None = None
    subject: str | None = None

@tool("create_draft", args_schema=CreateDraftInput)
def create_draft(thread_id: str, body_markdown: str,
                 style_profile_id: str | None = None,
                 subject: str | None = None) -> dict:
    """Create a reply draft in a thread. Does NOT send. Reversible."""
    html_body = render_styled_html(body_markdown, style_profile_id)
    return gmail_client.create_draft(thread_id, html_body, subject)

class SendEmailInput(BaseModel):
    draft_id: str
    confirmation_token: str = Field(..., description="Token issued by permission layer after approval")

@tool("send_email", args_schema=SendEmailInput)
def send_email(draft_id: str, confirmation_token: str) -> dict:
    """Send a previously created draft. IRREVERSIBLE. Requires a valid confirmation_token issued by the permission gate."""
    permission_service.verify_token(confirmation_token, action="send_email", resource=draft_id)
    return gmail_client.send_draft(draft_id)

class CreateEventInput(BaseModel):

```

```

title: str
start_iso: str
end_iso: str
attendees: list[str] = []
confirmation_token: str

@tool("create_event", args_schema=CreateEventInput)
def create_event(title: str, start_iso: str, end_iso: str,
                 attendees: list[str], confirmation_token: str) -> dict:
    """Create a Google Calendar event. IRREVERSIBLE. Requires confirmation_token."""
    permission_service.verify_token(confirmation_token, action="create_event",
    resource=title)
    return calendar_client.create_event(title, start_iso, end_iso, attendees)

```

6.2 Full Tool Catalog

Tool	Side Effect	Gated?
list_emails / search_emails	Read-only	No
get_thread	Read-only	No
get_attachment	Read-only	No
apply_label / categorize	Reversible	No (configurable)
create_draft	Reversible	No
update_draft	Reversible	No
send_email	Irreversible	Yes — always
check_availability	Read-only	No
create_event	Irreversible	Yes — always
update_event / cancel_event	Irreversible	Yes — always
create_task / create_reminder	Reversible	Configurable
get_style_profile	Read-only	No

7. Permission & Approval System

This is the most safety-critical subsystem. It determines whether an agent's proposed action executes immediately, waits for human approval, or is blocked outright.

7.1 Permission Model

Permissions are defined per user, per action type, and optionally per condition (e.g. "auto-approve send_email only to contacts in my known-contacts list"). Three authorization levels:

Level	Behavior	Default For
AUTO	Executes immediately, no human step. Logged for audit.	list_emails, get_thread, categorize, summarize
CONFIRM	Action is prepared (e.g. draft created) and queued in dashboard; executes only after explicit user approval.	send_email, create_event, update_event
BLOCKED	Agent may not propose this action at all under current rules.	Bulk delete, forwarding to external unknown domains (configurable)

7.2 Approval Queue & Confirmation Tokens

When a worker agent produces an action requiring CONFIRM, the Permission Layer writes a row to the approval_queue table, generates a single-use confirmation_token scoped to (user, action, resource, expiry), and pushes a real-time event to the dashboard via WebSocket. The graph execution interrupts at that node using LangGraph's `interrupt()` primitive.

```
def permission_gate_node(state: MailAgentState) -> MailAgentState:
    for action in state["pending_approvals"]:
        level = policy_engine.classify(
            user_id=state["user_id"],
            action_type=action["type"],
            resource=action["resource"],
        )
        if level == "AUTO":
            action["status"] = "approved"
        elif level == "CONFIRM":
            approval = approval_queue_repo.create(
                user_id=state["user_id"], action=action
            )
            notify_dashboard(state["user_id"], approval)
            # graph pauses here until /approvals/{id}/approve or /reject is called
            interrupt({"approval_id": approval.id, "action": action})
        elif level == "BLOCKED":
            action["status"] = "blocked"
            state["errors"].append({"action": action, "reason": "blocked_by_policy"})
    return state
```

7.3 Dashboard Approval UX

- Pending approvals appear in a persistent "Needs your review" panel — never auto-dismissed.
- Each approval shows: the exact email/event content, the recipient(s), the agent's reasoning, and one-click Approve / Edit / Reject.
- "Edit" lets the user modify the draft body or event details before approving — edits are saved back before the token is issued.
- Users can set per-contact or per-label auto-approve rules (e.g. "always auto-send replies to my recurring newsletter unsubscribe requests") — but `send_email` and `create_event` remain CONFIRM by default and require explicit opt-in to relax.

7.4 Audit Logging

Every tool call — whether AUTO or CONFIRM — writes an immutable row to `audit_log` with: timestamp, `user_id`, agent name, tool name, input parameters, output, the `approval_id` if applicable, and the LLM's stated reasoning for the action. This is non-negotiable for trust and debugging.

8. Database Schema (PostgreSQL)

8.1 Core Tables

```

-- Users & OAuth credentials
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT UNIQUE NOT NULL,
  display_name TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE oauth_credentials (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  provider TEXT NOT NULL CHECK (provider IN ('google','microsoft')),
  access_token_encrypted BYTEA NOT NULL,
  refresh_token_encrypted BYTEA NOT NULL,
  scopes TEXT[] NOT NULL,
  expires_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Style / formatting profiles
CREATE TABLE style_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  name TEXT NOT NULL,                -- e.g. "Formal client", "Quick internal"
  signature_html TEXT,
  font_family TEXT DEFAULT 'Arial',
  font_size INT DEFAULT 11,
  accent_color TEXT,                 -- hex, used for headers/links in HTML body
  tone TEXT DEFAULT 'neutral',       -- formal | casual | neutral | match_sender
  is_default BOOLEAN DEFAULT false
);

-- Cached email metadata (not full bodies, to limit PII at rest -- see Sec. 15)
CREATE TABLE email_cache (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  provider_message_id TEXT NOT NULL,
  thread_id TEXT NOT NULL,
  sender TEXT, subject TEXT,
  snippet TEXT,
  category TEXT,
  received_at TIMESTAMPTZ,
  UNIQUE(user_id, provider_message_id)
);

-- Drafts created by agents (pending or sent)
CREATE TABLE drafts (

```

```

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    thread_id TEXT NOT NULL,
    provider_draft_id TEXT,
    body_markdown TEXT,
    body_html TEXT,
    style_profile_id UUID REFERENCES style_profiles(id),
    status TEXT DEFAULT 'pending' CHECK (status IN
('pending', 'approved', 'sent', 'rejected')),
    created_by_agent TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- Approval queue
CREATE TABLE approval_queue (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    action_type TEXT NOT NULL,          -- send_email | create_event | ...
    resource_id TEXT NOT NULL,
    payload JSONB NOT NULL,
    agent_reasoning TEXT,
    status TEXT DEFAULT 'pending' CHECK (status IN
('pending', 'approved', 'rejected', 'expired')),
    confirmation_token TEXT,
    expires_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT now(),
    resolved_at TIMESTAMPTZ
);

-- Permission rules
CREATE TABLE permission_rules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    action_type TEXT NOT NULL,
    condition JSONB,                    -- e.g. {"recipient_domain": "mycompany.com"}
    level TEXT NOT NULL CHECK (level IN ('AUTO', 'CONFIRM', 'BLOCKED')),
    created_at TIMESTAMPTZ DEFAULT now()
);

-- Reminders / tasks
CREATE TABLE reminders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    related_thread_id TEXT,
    title TEXT NOT NULL,
    due_at TIMESTAMPTZ,
    status TEXT DEFAULT 'open' CHECK (status IN ('open', 'done', 'dismissed')),

```

```

    created_at TIMESTAMPTZ DEFAULT now()
);

-- Calendar events created by the system (mirror of provider event)
CREATE TABLE calendar_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    provider_event_id TEXT,
    title TEXT, start_at TIMESTAMPTZ, end_at TIMESTAMPTZ,
    attendees TEXT[],
    source_thread_id TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- Immutable audit log
CREATE TABLE audit_log (
    id BIGSERIAL PRIMARY KEY,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    agent_name TEXT NOT NULL,
    tool_name TEXT NOT NULL,
    input_params JSONB,
    output JSONB,
    approval_id UUID REFERENCES approval_queue(id),
    reasoning TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- LangGraph checkpoints (state persistence for interrupts/resume)
-- Created automatically by langgraph-checkpoint-postgres; do not hand-author.

```

8.2 Indexing Notes

- Index `audit_log(user_id, created_at DESC)` for dashboard activity feeds.
- Index `approval_queue(user_id, status)` for the live pending-approvals panel.
- Index `email_cache(user_id, thread_id)` and `(user_id, category)` for fast categorized views.

9. Backend Architecture

9.1 Service Breakdown

Service	Responsibility	Tech
API Gateway	Auth, request routing, WebSocket connections for live updates	FastAPI
Agent Orchestrator	Runs the LangGraph graph per request/conversation	LangGraph + Claude API
Integration Adapters	Provider-specific clients (Gmail, Google Calendar; later Outlook/Graph)	Google API Python Client, MSAL
Permission Service	Policy evaluation, token issuance/verification	Python + Postgres
Background Worker	Polling for new emails, scheduled summaries, reminder triggers	Celery + Redis, or APScheduler for prototype
Notification Service	Pushes approval requests and agent updates to dashboard	WebSockets / Server-Sent Events

9.2 New Email Trigger Flow (Background Worker)

1. Worker polls Gmail history API (or uses Gmail Push Notifications via Google Cloud Pub/Sub for near-real-time updates) on a per-user schedule.
2. New messages are written to `email_cache`.
3. If the user has configured "auto-categorize new mail," a lightweight graph run is triggered with only the Categorizer agent active.
4. If a category matches a user-defined rule (e.g. "meeting requests → notify me immediately"), a dashboard notification fires.

Note: Use Gmail Push Notifications (Pub/Sub) instead of polling once you're past prototype — polling burns API quota and adds latency. For the fast prototype, a simple 60-second poll loop with the history API is fine.

9.3 Why FastAPI + LangGraph (and not a single monolith prompt)

- FastAPI gives you async request handling, WebSocket support, and easy OpenAPI docs for the dashboard frontend to consume.
 - LangGraph gives durable execution: if the backend restarts mid-approval-wait, the checkpointers resumes the exact graph state rather than losing the user's in-flight request.
 - Decoupling orchestration (LangGraph) from transport (FastAPI) means you can later expose the same agent graph via a CLI, Slack bot, or Outlook plugin without rewriting agent logic.
-

10. Frontend Dashboard Specification

10.1 Core Views

View	Purpose
Chat / Command Panel	Primary interaction surface — type instructions, see agent responses, inline drafts
Approval Queue	Live list of actions awaiting confirmation, with Approve/Edit/Reject
Inbox Overview	Categorized email list, synced from email_cache, filterable by label/date/sender
Activity / Audit Feed	Chronological log of everything agents have done, with reasoning shown on expand
Style Profiles	Manage signatures, tone presets, fonts/colors per context (e.g. "Client-facing" vs "Internal")
Permission Rules	Configure AUTO / CONFIRM / BLOCKED per action type and condition
Calendar View	Visual calendar showing events the system created or proposed, with conflict highlighting
Agent Status Panel	Shows which agents are currently active/running, useful for transparency during parallel execution

10.2 Real-Time Updates

The dashboard maintains a WebSocket connection per user session. Agent state changes (task started, task completed, approval needed, error) push immediately rather than relying on polling, so a user watching the dashboard sees the multi-agent system working in near real time — this is also good for building trust in an autonomous system.

10.3 Suggested Stack

- React + TypeScript, Vite for build tooling

- TanStack Query for server state, native WebSocket hook for live events
 - Tailwind CSS for styling — fast to iterate for a prototype dashboard
 - A component library such as shadcn/ui for forms, dialogs (approval modals), and tables (audit log, inbox)
-

11. Gmail + Google Calendar Integration (Phase 1)

11.1 OAuth Setup

1. Create a project in Google Cloud Console, enable Gmail API, Google Calendar API, and Google Tasks API.
2. Configure OAuth consent screen (Internal if G Suite org, External + verification if public product).
3. Create OAuth 2.0 Client ID (Web application type), set redirect URI to your backend callback (e.g. <https://yourapp.com/auth/google/callback>).
4. Request the minimum necessary scopes — see table below.

11.2 Required Scopes

Scope	Purpose
gmail.readonly	Reading emails, threads, search
gmail.compose	Creating drafts (does not allow sending)
gmail.send	Sending approved drafts
gmail.labels	Reading and applying categorization labels
calendar.events	Creating/reading/updating calendar events
calendar.readonly	Checking availability/conflicts without write access, where send isn't needed
tasks	Creating reminders via Google Tasks (or use a custom reminders table instead — see 8.1)

Note: Request gmail.compose and gmail.send as separate, distinct scope grants in your consent screen copy — being explicit that "drafting" and "sending" are different permissions builds user trust and mirrors the product's own permission model.

11.3 Key API Patterns

```
# Reading
service.users().messages().list(userId='me', q='is:unread from:client.com').execute()
service.users().threads().get(userId='me', id=thread_id, format='full').execute()

# Drafting (reversible)
draft_body = {'message': {'raw': base64_encoded_mime, 'threadId': thread_id}}
service.users().drafts().create(userId='me', body=draft_body).execute()

# Sending (irreversible -- only after permission gate clears)
service.users().drafts().send(userId='me', body={'id': draft_id}).execute()

# Labels (categorization)
service.users().messages().modify(
    userId='me', id=msg_id,
    body={'addLabelIds': [label_id]}
).execute()
```

```
# Google Calendar - conflict check before proposing a meeting
freebusy = calendar_service.freebusy().query(body={
    "timeMin": start_iso, "timeMax": end_iso,
    "items": [{"id": "primary"}]
}).execute()

# Creating an event (irreversible -- gated)
event = calendar_service.events().insert(
    calendarId='primary',
    body={
        "summary": title,
        "start": {"dateTime": start_iso},
        "end": {"dateTime": end_iso},
        "attendees": [{"email": a} for a in attendees],
    },
    sendUpdates="all"
).execute()
```

11.4 Real-Time Sync (Production)

Use Gmail's `watch()` API with Google Cloud Pub/Sub to receive push notifications on mailbox changes, rather than polling. Set up a Pub/Sub topic, grant Gmail publish rights to it, and subscribe your backend to receive history IDs, then call `history.list()` to fetch the delta.

12. Phase 2: Microsoft Outlook / 365 Migration Plan

Because the architecture treats Gmail as one implementation of an abstract MailProvider interface, Phase 2 is primarily an adapter-writing exercise, not a redesign.

12.1 Provider Abstraction

```
from abc import ABC, abstractmethod

class MailProvider(ABC):
    @abstractmethod
    def list_messages(self, query: str, max_results: int) -> list[Message]: ...
    @abstractmethod
    def get_thread(self, thread_id: str) -> Thread: ...
    @abstractmethod
    def create_draft(self, thread_id: str, html_body: str, subject: str | None) ->
    Draft: ...
    @abstractmethod
    def send_draft(self, draft_id: str) -> SendResult: ...
    @abstractmethod
    def apply_label(self, message_id: str, label: str) -> None: ...

class CalendarProvider(ABC):
    @abstractmethod
    def check_availability(self, start: str, end: str) -> bool: ...
    @abstractmethod
    def create_event(self, title: str, start: str, end: str, attendees: list[str]) ->
    Event: ...

class GmailProvider(MailProvider): ... # Phase 1
class GoogleCalendarProvider(CalendarProvider): ... # Phase 1
class OutlookProvider(MailProvider): ... # Phase 2
class OutlookCalendarProvider(CalendarProvider): ... # Phase 2
```

Worker agents and tools call the abstract interface, not GmailProvider directly. The active provider is resolved per-user based on which account they connected, so a single deployment can serve both Gmail and Outlook users simultaneously once Phase 2 ships.

12.2 Outlook-Specific Notes

- Auth: Microsoft Entra ID (Azure AD) app registration, OAuth 2.0 via MSAL, using Microsoft Graph API.
- Scopes: Mail.Read, Mail.ReadWrite (drafts), Mail.Send, Calendars.ReadWrite.
- Mail API: Microsoft Graph /me/messages, /me/mailFolders for categorization (Outlook uses folders + categories rather than Gmail labels — map your internal Category model to Outlook's "categories" field).
- Real-time sync: Microsoft Graph webhooks (subscriptions) instead of Gmail Pub/Sub push.
- HTML rendering differences: Outlook's rendering engine (Word-based) has known CSS quirks (e.g. limited flexbox/grid support) — the formatting engine (Section 13) needs an Outlook-safe HTML template variant, typically table-based layout.

13. Formatting & Style Engine

This directly addresses the requirement that the agent follow exact formatting commands — signature, font, color, indentation, spacing — reliably rather than approximately.

13.1 Structured Style Object

The Drafter agent never freehand-generates raw HTML styling from scratch per request. Instead, natural-language formatting instructions are parsed into a structured StyleSpec, which is then deterministically rendered to HTML/CSS. This keeps formatting consistent and testable.

```
class StyleSpec(BaseModel):
    tone: Literal["formal", "casual", "neutral", "match_sender"] = "neutral"
    font_family: str = "Arial"
    font_size_pt: int = 11
    text_color: str = "#000000"
    accent_color: str | None = None          # used for headers/links
    line_spacing: float = 1.15
    paragraph_indent_px: int = 0
    include_signature: bool = True
    signature_profile_id: str | None = None
    bullet_style: Literal["dash", "dot", "numbered"] = "dot"
```

13.2 Parsing Instructions into StyleSpec

A dedicated lightweight LLM call (or, for the prototype, simple rule-based parsing) converts free text like "make it formal, blue headers, add my signature, use Calibri" into a StyleSpec object before drafting begins:

```
STYLE_PARSE_PROMPT = """Extract formatting instructions from the user's request into
the StyleSpec schema. If a field isn't mentioned, omit it (defaults apply).
Only extract explicit or strongly implied formatting preferences – do not invent
preferences the user didn't state.
```

```
User request: {instruction}
"""
```

```
style_spec = claude_client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=300,
    tools=[style_spec_tool],          # forces structured tool-call output
    messages=[{"role": "user", "content":
STYLE_PARSE_PROMPT.format(instruction=instruction)}]
)
```

13.3 Deterministic Rendering

Markdown body content from the Drafter agent + StyleSpec are passed to a template renderer (e.g. Jinja2) that produces final HTML email markup — never letting the LLM hand-write the closing HTML/CSS directly, which avoids inconsistent or broken markup.

```
def render_styled_html(body_markdown: str, style: StyleSpec) -> str:
    body_html = markdown_to_html(body_markdown)
    signature_html = get_signature(style.signature_profile_id) if
style.include_signature else ""
    return jinja_env.get_template("email_base.html.j2").render(
        body=body_html,
        font_family=style.font_family,
        font_size=style.font_size_pt,
        color=style.text_color,
        accent=style.accent_color or "#1F4E79",
        line_height=style.line_spacing,
        indent=style.paragraph_indent_px,
        signature=signature_html,
    )
```

Note: Maintain two template variants: one standard HTML/CSS template, and one Outlook-safe table-based template for Phase 2, since Outlook's rendering engine does not support modern CSS layout reliably.

13.4 Signature Management

Signatures are stored as pre-approved HTML snippets per `style_profiles` row (Section 8.1), not regenerated by the LLM each time. This guarantees the signature is always exactly what the user configured — name, title, contact details, logo — with zero risk of the model altering it.

14. Observability, Testing & Evals

14.1 Tracing

- Use LangSmith (or an OpenTelemetry-compatible alternative) to trace every graph run: which nodes executed, in what order, with what inputs/outputs, and latency per step.
- Tag traces with `user_id` and instruction so you can replay a specific user's request when debugging an issue they report.

14.2 Agent-Level Evals

Agent	Eval Method
Categorizer	Labeled test set of emails with ground-truth categories; measure precision/recall per category
Summarizer	Human-graded rubric (faithfulness, completeness, conciseness) on a sample set; consider an LLM-as-judge pass with periodic human spot-checks
Drafter	Style-adherence checks (does output match requested StyleSpec exactly) + tone/quality human review
Scheduler	Conflict-detection accuracy against synthetic calendar fixtures with known overlaps
Supervisor	Task-decomposition correctness — does a given instruction get routed to the right agent set

14.3 Integration & Safety Tests

- Permission layer: automated tests asserting that `send_email` and `create_event` cannot execute without a valid `confirmation_token`, including expired/reused token rejection.
- Idempotency: re-running an interrupted graph after backend restart must not double-send an email or double-book a calendar slot.
- Rate limiting / quota tests against Gmail and Calendar API limits, with backoff behavior verified.

15. Security & Compliance

- Encrypt OAuth tokens at rest (e.g. AES-256 via a KMS-managed key) — never store raw access/refresh tokens in plaintext, per the `oauth_credentials` schema in Section 8.
- Minimize email body retention: cache only metadata/snippets by default (Section 8.1); fetch full bodies on demand rather than persisting them, to reduce PII exposure at rest.
- Scope requests to the minimum needed (Section 11.2) and clearly explain each scope's purpose on the consent screen.

- All `confirmation_tokens` are single-use, short-lived (recommend 15-minute expiry), and scoped to one specific action + resource — they cannot be replayed.
 - Implement per-user rate limits on agent-initiated sends to contain damage from a misconfigured rule or compromised session.
 - Log access to the `audit_log` table itself — who viewed whose audit history — since it contains sensitive behavioral data.
 - If targeting EU or regulated users, plan for GDPR data export/deletion endpoints early; the schema's `user_id` foreign keys with `ON DELETE CASCADE` make full account deletion straightforward.
-

16. Build Roadmap

Milestone 1 — Foundation (Week 1-2)

- Google Cloud project, OAuth flow, FastAPI skeleton, Postgres schema deployed
- Single-agent prototype: read + categorize only, no LangGraph yet — validate Gmail API integration end to end

Milestone 2 — Multi-Agent Core (Week 3-4)

- Introduce LangGraph: Supervisor + Reader + Categorizer + Summarizer agents
- Basic chat UI wired to the graph via FastAPI WebSocket

Milestone 3 — Drafting & Style Engine (Week 5)

- Drafter agent, StyleSpec parsing, Jinja2 HTML rendering, signature management
- `create_draft` tool fully working, visible in dashboard

Milestone 4 — Permission Layer & Sending (Week 6)

- `approval_queue`, confirmation tokens, `permission_rules`, dashboard approval UX
- `send_email` gated end-to-end — this is the first irreversible action in the system, test heavily

Milestone 5 — Calendar & Reminders (Week 7)

- Scheduler and Reminder agents, conflict checking, create_event gated flow

Milestone 6 — Dashboard Polish & Observability (Week 8)

- Full dashboard views (Section 10), LangSmith tracing, audit feed UI
- Agent-level evals against labeled test sets (Section 14.2)

Milestone 7 — Outlook Adapter (Phase 2, Week 9+)

- MailProvider/CalendarProvider abstraction already in place from Milestone 1 if built correctly — implement OutlookProvider and OutlookCalendarProvider
 - Outlook-safe HTML template variant for the style engine
-

17. Appendix: Technology Stack Summary

Layer	Technology
Agent Orchestration	LangGraph + LangChain
LLM	Claude (Anthropic API) — Sonnet for most agents, consider Haiku for high-volume categorization to control cost
Backend API	FastAPI (Python 3.11+)
Background Jobs	Celery + Redis (or APScheduler for prototype)
Database	PostgreSQL (+ langgraph-checkpoint-postgres for graph state)
Cache / Queue	Redis
Frontend	React + TypeScript, Vite, Tailwind CSS, shadcn/ui
Real-time	WebSockets (FastAPI native or Socket.IO)
Email Provider (Phase 1)	Gmail API + Google Calendar API + Google Tasks API
Email Provider (Phase 2)	Microsoft Graph API (Outlook Mail + Calendar)
Tracing / Evals	LangSmith
Templating	Jinja2 (HTML email rendering)
Auth	OAuth 2.0 (Google), MSAL (Microsoft)
Secrets / Encryption	Cloud KMS (GCP) or equivalent for token encryption at rest

End of document.